

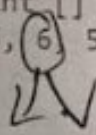
Question 1 [CO2] [10 Points]

[Answer on the answer-script]

Suppose you are given an array `arr` of size `n`, where `n` will always be an odd number. A position `X` in `arr` is considered a **Lucky Position** if it satisfies the following conditions:

- The product of all elements before position `X` is equal to the product of all elements after position `X`
- The sequence of signs (positive/negative) of elements before position `X` is the mirror image of the sequence of signs after position `X`. If there are no elements on one side, its product is considered 1.

Write a Java program to determine the **first Lucky Position** in the array. If such a position exists, print "Lucky Position: `X`" and "Value: `Y`" (where `X` is the position and `Y` is the element at that position); otherwise, print "No Lucky Position". [Your code should work for any given odd-length integer array] [You are not allowed to use any built-in methods or attributes other than `.length`]

Given Array:	Sample Output	Explanation:
<pre>int [] arr = {-3, -2, 5, 6, 5, -2, -3};</pre> 	<pre>Lucky Position: 4 Value: 6</pre>	Position 4 (value = 6): Left product = $-3 \times -2 \times 5 = 30$ Right product = $5 \times -2 \times -3 = 30$ Left signs (pos 4 to left): $\{5, -2, -3\} \rightarrow (+, -, -)$ Right signs (pos 4 to right): $\{5, -2, -3\} \rightarrow (+, -, -)$ Sign patterns match $\rightarrow$ Lucky Position: 4
<pre>int [] arr = {5, 1, -3, 2, 9, 2, 3, 1, 5};</pre>	<pre>No Lucky Position</pre>	Position 5 (value = 9): Left product = $5 \times 1 \times -3 \times 2 = -30$ Right product = $2 \times 3 \times 1 \times 5 = 30$ Products are not equal $\rightarrow$ fails No valid position found in arr.

Question 2 [CO2] [10 Points]

[Answer on the answer-script]

You are a junior developer at a galactic data processing firm where remote satellites send encoded computation strings containing letters, digits, and operators (+, -, *). Your task is to scan the string and locate each operator one by one. For every operator, check whether at least two digits appear before it and at least two digits appear after it. If the condition is satisfied, extract the last two digits before the operator to form one two-digit number and the first two digits after the operator to form another two-digit number, then perform the operation and print the result. If the condition is not satisfied, print Invalid Signal for that operator. You are not allowed to use any built-in methods except `.charAt()`, `.length()`, `.nextLine()` and `.equals()`.

Sample Input	Sample Output	Explanation:
<u>ab12c*d34e+45e*2ui-1</u>	Result: <u>408</u> Result: 79 Invalid Signal Invalid Signal	For each operator, only the characters between the previous and next operators are checked. The first * operator finds 12 before it and 34 after it, so $12 \times 34 = 408$. The + operator finds 34 and 45, so $34 + 45 = 79$. The second * has only one digit (2) after it before the next operator, so it is Invalid Signal. The - operator has only one digit on each side (2 and 1), so it is also Invalid Signal.
52ba1298a+76b*54c-3d	Result: 174 Result: 4104 Invalid Signal	The first + operator finds <u>98</u> (last two digits) before it and 76 after it, so $98 + 76 = 174$. The * operator finds 76 before it and 54 after it, so $76 \times 54 = 4104$. The - operator has only one digit (3) after it, so it is Invalid Signal.

Question 3 [CO1] [5 Points]

[Answer on the question paper]

This is a modified selection sort algorithm that sorts only the even elements in ascending order while leaving the odd elements unchanged in their original positions. The comparison and swapping are restricted to even numbers only. For example, in [20, 25, 3, 8, 10], the even elements 20, 8, 10 are sorted into 8, 10, 20 and placed back in their positions, while odd numbers remain unchanged, resulting in [8, 25, 3, 10, 20].

```

1 import java.util.Arrays;
2 public class EvenSort{
3     public static void sortEven (int array){
4         for (int i = 0; i < array.length - 1; i++){
5             if (array[i] % 2 != 0) {
6                 break;
7             }
8             int min_index = 0;
9             for (int j = i + 1; j < array.length; j++) {
10                 if ((array[j] % 2 == 0) && array[j] > array[min_index]){
11                     min_index = j;
12                 }
13             }
14             if (array[min_index] % 2 == 0 && array[i] % 2 == 0){
15                 int temp = array[i];
16                 array[min_index] = array[i];
17                 array[i] = temp;
18             }
19         }
20     }
21     public void main(String [] args) {
22         int[] array = {20, 25, 10, 8, 3};
23         sortEven(array);
24         System.out.println(Arrays.toString(array));
25     }
26 }

```

There are 6 errors in the above code. Write your corrections in the table given below. For better understanding, one error correction is shown.

Line Number	Fix
Line 21	public static void main(String [] args){
Line 16	array[min_index] = array[i];
Line 17	array[i] = temp;
Line 9	for (int j = 0; j < array.length - i - 1; j++) {
Line 14	if (array[min_index] % 2 == 0 && array[i] % 2 == 0) {
Line 5	int temp = array[i];

Question 4 [CO3] [5 Points]
[Answer on the question paper]

Illustrate the outputs of the following statements. Provide your workings the answer script to verify your outputs. Your answer will not be accepted without the workings. All of the outputs must be in the question paper.

```

1 public class FinalA{
2     public static void main(String[] args){
3         int[] arr = {4, 7, 2, 9, 5, 3};
4         System.out.println(metB(arr,1));
5     }
6     public static int metB(int[] arr, int index){
7         if (index >= arr.length-1) {
8             return arr[index]++;
9         }
10        int a = metB(arr, index + 1);
11        int c = metA(arr[index+1], a, 3);
12        System.out.println(a+c);
13        return c - (++index);
14    }
15    public static int metA(int num, int x, int y){
16        if (num % 2 == 0) {
17            return x + (num * y);
18        } else {
19            return (x * y) - num;
20        }
21    }
22 }
    
```

Output

19

22

25

✓

Time: 100 minutes

Computer Science and Engineering
Final Examination Spring 2026
CSE 110: Programming Language I

B

- ✓ Write your Set Number on top of the answer script in LARGE FONT.
- ✓ Answer all questions. Use back part of the answer script for rough work.
- ✓ Answer Question 1 & 2 at the beginning part of the answer script.
- ✓ Figure in bracket [] next to each question indicates marks for that question.
- ✓ At the end of exam, put question paper inside answer script and return both.
- ✓ Understanding the question is part of the exam, please do not ask questions

No washroom breaks

Section: _____ ID: _____ Name in CAPITAL: _____

Question 1 [CO2] [10 Points]
[Answer on the answer-script]

Suppose you are given an array `arr` of size `n`, where `n` will always be an odd number. A position `X` in `arr` is considered a **Lucky Position** if it satisfies the following conditions:

- The product of all elements before position `X` is equal to the product of all elements after position `X`
- The sequence of signs (positive/negative) of elements before position `X` is the mirror image of the sequence of signs after position `X`. If there are no elements on one side, its product is considered 1.

Write a Java program to determine the first Lucky Position in the array. If such a position exists, print "Lucky Position: `X`" and "Value: `Y`" (where `X` is the position and `Y` is the element at that position); otherwise, print "No Lucky Position". [Your code should work for any given odd-length integer array] [You are not allowed to use any built-in methods or attributes other than `.length`]

Given Array:	Sample Output	Explanation:
<code>int [] arr = {-3, -2, 5, 6, 5, -2, -3};</code>	Lucky Position: 4 Value: 6	Position 4 (value = 6): Left product = $-3 \times -2 \times 5 = 30$ Right product = $5 \times -2 \times -3 = 30$ Left signs (pos 4 to left): $\{5, -2, -3\} \rightarrow (+, -, -)$ Right signs (pos 4 to right): $\{5, -2, -3\} \rightarrow (+, -, -)$ Sign patterns match → Lucky Position: 4
<code>int [] arr = {5, 1, -3, 2, 9, 2, 3, 1, 5};</code>	No Lucky Position	Position 5 (value = 9): Left product = $5 \times 1 \times -3 \times 2 = -30$ Right product = $2 \times 3 \times 1 \times 5 = 30$ Products are not equal → fails No valid position found in arr.

Question 2 [CO2] [10 Points]

[Answer on the answer-script]

You are a junior developer at a galactic data processing firm where remote satellites send encoded computation strings containing letters, digits, and operators (+, -, *). Your task is to scan the string and locate each operator one by one. For every operator, check whether at least two digits appear before it and at least two digits appear after it. If the condition is satisfied, extract the last two digits before the operator to form one two-digit number and the first two digits after the operator to form another two-digit number, then perform the operation and print the result. If the condition is not satisfied, print Invalid Signal for that operator. You are not allowed to use any built-in methods except `.charAt()`, `.length()`, `.nextline()` and `.equals()`.

Sample Input	Sample Output	Explanation:
ab12c*d34e+45e*2ui-1	Result: 408 Result: 79 Invalid Signal Invalid Signal	For each operator, only the characters between the previous and next operators are checked. The first * operator finds 12 before it and 34 after it, so $12 \times 34 = 408$. The + operator finds 34 and 45, so $34 + 45 = 79$. The second * has only one digit (2) after it before the next operator, so it is Invalid Signal. The - operator has only one digit on each side (2 and 1), so it is also Invalid Signal.
52ba1298a+76b*54c-3d	Result: 174 Result: 4104 Invalid Signal	The first + operator finds 98 (last two digits) before it and 76 after it, so $98 + 76 = 174$. The * operator finds 76 before it and 54 after it, so $76 \times 54 = 4104$. The - operator has only one digit (3) after it, so it is Invalid Signal.

Question 3 [CO1] [5 Points]

[Answer on the question paper]

This is a modified selection sort algorithm that sorts only the odd elements in ascending order, while leaving the even elements unchanged in their original positions. The comparison and swapping are restricted to odd numbers only. For example, in [20, 25, 10, 9, 3], the odd elements 25, 9, 3 are sorted into 3, 9, 25 and placed back in their positions, while even numbers remain unchanged, resulting in [20, 3, 10, 9, 25].

```

1 import java.util.Arrays;
2 public class OddSort {
3     public static void sortOdd (String[] array) {
4         for (int i = 0; i < array.length - 1; i++) {
5             if (array[i] % 2 != 0) {
6                 continue;
7             }
8             int min index = i;
9             for (int j = i + 1; j < array.length; j++) {
10                 if (array[j] % 2 != 0 || array[j] < array[min index]){
11                     min index = j;
12                 }
13             }
14             if (array[min index] % 2 != 0 && array[i] % 2 != 0) {
15                 int temp = array[min index];
16                 array[min index] = array[i];
17                 array[i] = temp;
18             }
19         }
20     }
21     public void main(String [] args) {
22         int[] array = {20, 25, 10, 9, 3};
23         sortOdd(array);
24         System.out.println(Arrays.toString(array));
25     }
26 }

```

There are 6 errors in the above code. Write your corrections in the table given below. For better understanding, one error correction is shown.

Line Number	Fix
Line 21	public static void main(String [] args)
Line 3	public static void sortOdd(int [] array) {
Line 4	for (int i = 0; i < array.length; i++) {
Line 5	if (array[i] % 2 == 0) {
Line 9	for (int j = i + 1; j < array.length - 1; j++) {
Line 11	min-index = j;

Question 4 [CO3] [5 Points]

[Answer on the question paper]

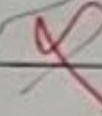
Illustrate the outputs of the following statements. Provide your workings on the answer script to verify your outputs. Your answer will not be accepted without the workings. All of the outputs must be in the question paper.

```
1 public class FinalB{
2     public static void main(String[] args){
3         int[] arr = {3, 2, 5, 4, 6, 7};
4         System.out.println(metB(arr,1));
5     }
6     public static int metB(int[] arr, int index){
7         if (index >= arr.length-1) {
8             return arr[index]++;
9         }
10        int a = metB(arr, index + 1);
11        int c = metA(arr[index+1], a, 3);
12        System.out.println(a+c);
13        return c - (++index);
14    }
15    public static int metA(int num, int x, int y){
16        if (num % 2 == 0) {
17            return x + (num * y);
18        } else {
19            return (x * y) - num;
20        }
21    }
22 }
```

Output

15

7.




```

public class q1{
    public static void main(String [] args){
        int [] arr= {-3,-2,5,6,5,-2,-3};
        boolean flag=false;
        for(int i=0;i<arr.length;i++){
            int product1=1;
            for(int j=i-1;j>=0;j--){
                product1*=arr[j];
            }
            int product2=1;
            for(int j=i+1;j<arr.length;j++){
                product2*=arr[j];
            }
            int j=i-1,k=i+1;
            boolean sign_flag=false;
            while(j>=0&& k<arr.length){
                if(!(arr[j]>0 && arr[k]>0) || (arr[j]<0 && arr[k]<0)){
                    sign_flag=true;
                    break;
                }
                j--; k++;
            }
            if(sign_flag) continue;
            if(product1==product2){
                flag=true;
                System.out.println("Lucky Position: "+(i+1));
                System.out.println("Value "+arr[i]);
                break;
            }
        }
    }
}

```

Class, Me

Lo

Le

Rig

Sig

On

```
,  
if(!flag){  
    System.out.println("No Lucky Position");  
}  
}}
```

```

import java.util.Scanner;
public class qs2{ public static void main(String [] args){ Scanner sc= new Scanner(System.in);
    String input= sc.next();
    int first_index=0,last_index=0;
    for(int i=0;i<input.length();i++){
        int count1=0, count2=0, num1=0, num2=0;
        while(last_index<input.length()-1){
            last_index++;
            if(input.charAt(last_index)=='+'||input.charAt(last_index)=='-'||input.charAt(last_index)=='*')break;
        }
        if(input.charAt(i)=='+'||input.charAt(i)=='-'||input.charAt(i)=='*'){
            for(int j=i-1;j>=first_index;j--){
                if(input.charAt(j)>='0' && input.charAt(j)<'9'){
                    if(count1==0){
                        num1+=input.charAt(j)-'0'; count1++;
                    }
                    else if(count1==1){
                        num1+=(input.charAt(j)-'0')*10; count1++;
                    }
                    else break;
                }
            }
            for(int j=i+1;j<last_index;j++){
                if(input.charAt(j)>='0' && input.charAt(j)<'9'){
                    if(count2==0){
                        num2+=(input.charAt(j)-'0')*10; count2++;
                    }
                    else if(count2==1){
                        num2+=input.charAt(j)-'0'; count2++;
                    }
                }
            }
        }
    }
}

```

Class, Method, Input

[0.2 X 5 = 1 mark]

Prev and Next Operator

[0.25 X 2 = 0.5 marks]

[1 mark]

Num 1&2 and 2 Digit

[0.25 X 4 = 1 mark]

Next Operator Find

[0.25 X 3 = 0.75 marks]

Operator check [1 mark]

Number 1

[5 X 0.2 = 1 mark]

Number 2

[5 X 0.2 = 1 mark]


```

        else break;
    } }
    if(count1==2 && count2==2){
        if(input.charAt(i)=='+') System.out.println("Result: "+(num1+num2));
        else if (input.charAt(i)=='-') System.out.println("Result: "+(num1-num2));
        else System.out.println("Result: "+(num1*num2));
    }
    else System.out.println("Invalid Signal");
    first_index=i;
}
}
}
}

```

2 Digit Validity

[0.5

Output

Output

Current Operator Find

Question 03:

Set A

Line Number	Corrected Code Line
Line 3	public static void sortEven (int[] array){
Line 6	continue
Line 8	int min_index = i ;
Line 10	if ((array[j] % 2 == 0) && array[j] < array[min_index]){
Line 15	int temp = array[min_index];

If any student changes Line 16,17 instead of Line 15, give 1 mark for both lines. No mark will be given if changes were made in either Line 16 or 17

Line 16: array[min_index] = temp;
Line 17: array[i] = array[min_index];

Set B

Line Number	Corrected Code Line
Line 3	public static void sortOdd (int[] array) {
Line 5	if (array[i] % 2 == 0) {
Line 10	if (array[j] % 2 != 0 && array[j] < array[min_index]){
Line 11	min_index = j ;
Line 16	array[min_index] = array[i] ;

Question 04:

<u>SET A</u>		<u>SET B</u>	
Serial	Output	Serial	Output
1	18	1	38



2	35	2	70
3	75	3	92
4	108	4	191
5	55	5	140